

KAMOTI : Key Alert and Monitoring for Online Tracking of Infrastructures

Project Proposal Draft

Submitted by:

Jimenez, Justine Lee

Ariola, Jon Luigi

Bauza, Xyrelle John

Caliliw, Marl Zeldrix

Conejos, Rainier Martin

Course: Web Systems and Technologies 2

Instructor: Ma'am N. Maylyn Bautista

Date of Submission: August 2026

Table of Contents

I. Problem Statement.....	2
II. Target Users.....	3
III. Proposed Features.....	3
Citizen-Facing Features.....	3
Staff-Facing Features.....	4
Administrator-Facing Features.....	4
Shared Features.....	4
IV. User Roles.....	5
V. System Architecture.....	5
VI. Initial ERD.....	8
Core Entities.....	9
VII. Technology Stack.....	11
References:.....	12

I. Problem Statement

The transition from manual reporting to a centralized digital system is a critical step in modernizing local traffic governance and policies. When local government units rely on disorganized channels like phone calls and walk-ins, the inefficiencies stack up, leading to unused resources and unsatisfied citizens. Beyond the disorganized systems, relying on them comes with a whole other set of deeper, complex systemic issues. Reports often come in from different channels, often disorganized, which makes real-time analysis difficult, especially if that process is also manual; this, in turn, wastes the LGUs' time instead of assisting with such cases in the real-world, and manpower is instead focused on the verification of such cases.

The implementation of a role-based Public Infrastructure Reporting Web Application does more than just digitize forms; it transforms how work is routed and executed. By integrating geo-tagging and visual evidence, citizens can use their mobile devices to drop a precise GPS pin and upload timestamped photos, eliminating location ambiguity and allowing dispatchers to assess damage severity before sending a crew. Implementing this kind of technology invokes civic

consciousness; it's often referred to as a "311 system" or crowdsourced civic issue reporting, and it yields benefits that extend far beyond simply fixing roads. By providing a public-facing dashboard where citizens can track the status of their report from submission to resolution, the LGU demonstrates transparency and accountability, which is a proven way to restore public trust and boost civic engagement.

Afonso (2017) notes that centralized web apps drastically lower the transaction costs of information gathering while highlighting the importance of closing the feedback loop to sustain citizen engagement. Similarly, Walwadkar et al. (2022) emphasize in their research that manual complaint systems fail under high volumes, outlining the necessity of implementing GPS sensors and visual proof to route issues effectively.

II. Target Users

User Group	Description
Citizens/Pedestrians (General Users)	Residents who want to report damaged infrastructure and track resolution progress.
Staff/Employees	Personnel responsible for reviewing, processing, and resolving reported issues (e.g., inspectors, maintenance coordinators).
Administrators	System managers responsible for overseeing users, categories, records, and overall reporting analytics.

III. Proposed Features

Citizen-Facing Features

- Account registration and login
- Submit infrastructure reports (category, description, photo, map location)

- Track status of submitted reports (Pending → Under Review → In Progress → Resolved)
- View public transparency board of all community reports
- Receive notifications on status updates

Staff-Facing Features

- View assigned or available reports/requests
- Update the status of reports as work progresses
- Add remarks, resolution notes, or completion photos
- Manage transactions related to processing a report (e.g., inspection logs, work orders)

Administrator-Facing Features

- Manage user accounts (citizens and staff)
- Manage infrastructure categories (roads, streetlights, drainage, signs, etc.)
- Manage and oversee all submitted records/reports
- Assign staff to reports
- Generate analytics and summary reports (volume by category, resolution time, status distribution)
- View system-wide activity logs

Shared Features

- Map-based geotagging of reports
- Photo upload/attachment support
- Notification system for status changes
- Public transparency board (view-only, no login required)

IV. User Roles

Role	Access Level	Core Responsibilities
Administrator	Full system access	Manage users, manage categories, manage records/reports, view analytics, assign staff
Staff/Employee	Restricted, operational access	Process assigned requests, update report status, manage related transactions/logs
General Users (Citizen/Pedestrians)	Public/registered access	Register an account, submit requests/bookings (reports), view status of own submissions

Role-Based Access Control (RBAC) will be implemented so that each role only sees the modules and data relevant to their function.

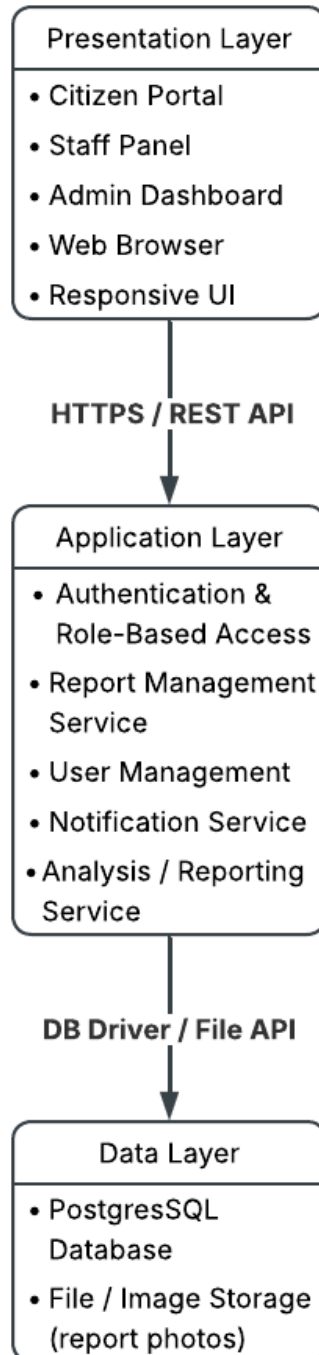
V. System Architecture

The system will follow a **three-tier web application architecture**, separating the system into presentation, application, and data layers. The **presentation layer** is what each of the three user roles interacts with directly citizens use a public-facing portal to register, submit reports with photos and map locations, and track status; staff use an operational panel to view assigned reports and update their progress and administrators use a dashboard to manage users, categories, and system-wide records. All three interfaces are served as a single responsive web application, with the UI conditionally rendering different views and permissions depending on the logged-in user's role. The public transparency board is the one exception. It's accessible without login, since its purpose is open visibility into community reports.

Beneath the presentation layer sits the **application layer**, which handles the actual business logic through a REST API. This includes an authentication service that issues and validates tokens (JWT), middleware that enforces role-based access control on every

request, a report management service that handles report creation, status updates, and assignment, and supporting services for notifications and analytics. This layer is what mediates all communication between the frontend and the database.

The **data layer** consists of the relational database (MySQL/PostgreSQL) storing users, categories, reports, and status history, along with a separate file/image storage mechanism for report photos. Third-party services are integrated at this level too which is a mapping API (Leaflet.js/OpenStreetMap or Google Maps) for geotagging report locations, and an email service like Nodemailer or Resend for notification delivery. This layered separation means each part of the system can be developed, tested, and scaled somewhat independently, let's say your team could swap the database engine or add a caching layer later without needing to rewrite the frontend or the API contracts between layers.

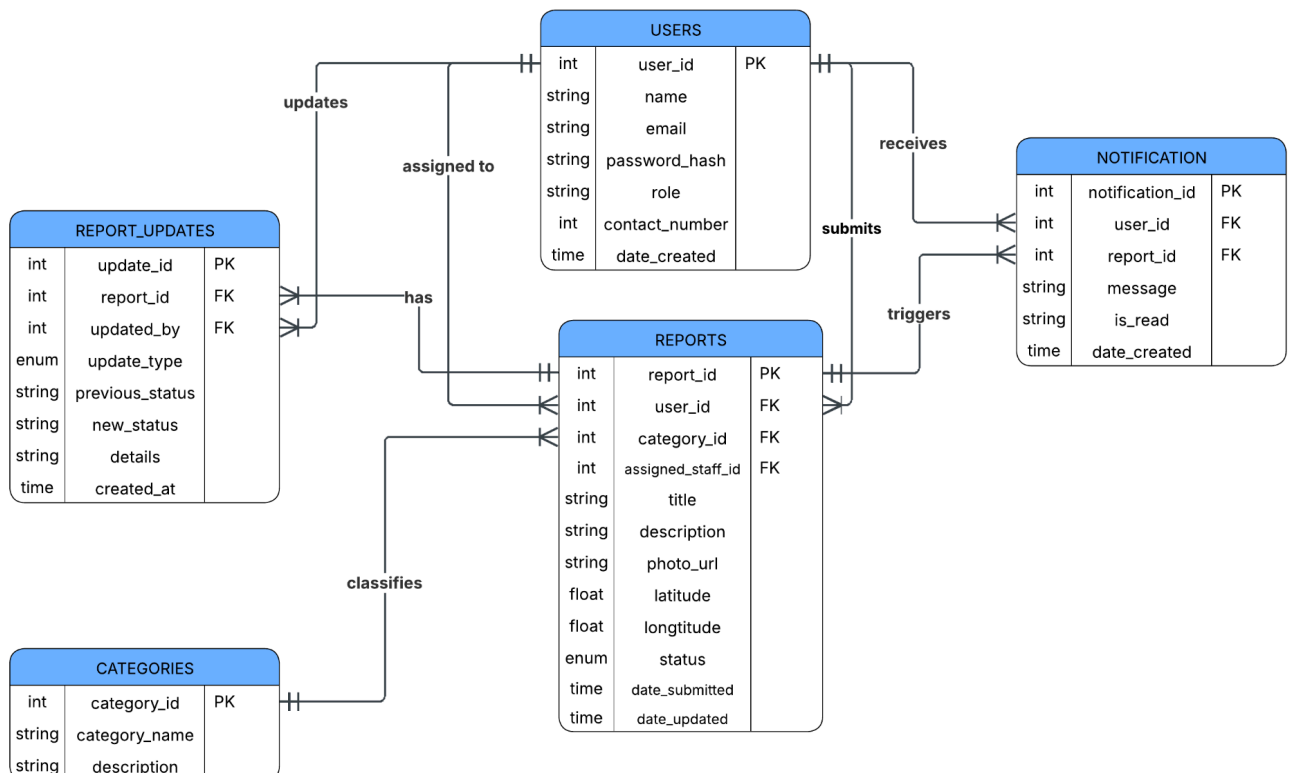


Key architectural notes:

- Authentication middleware will protect all restricted API endpoints. Role-based authorization will ensure that administrators, staff, and citizens can only perform actions allowed for their roles.

- The public transparency board will use a read-only API endpoint that does not require login. It will display only reviewed report information and will not expose citizens' personal information.
- Map/geotagging services will be integrated via a third-party API (e.g., Leaflet.js + OpenStreetMap, or Google Maps API).
- Report photos will be received and validated by the backend before being uploaded. The database will store the resulting file URL instead of the image file itself.
- When a report's status changes, the application will record the change in 'Reports' and create an in-app notification for the citizen who submitted the report.
- Only administrators can assign reports to staff. Staff can update reports assigned to them, while citizens can manage and view only their own submissions.

VI. Initial ERD



Core Entities

Users

- user_id (PK)
- name
- email
- password_hash
- role (Admin / Staff / Customer)
- contact_number
- date_created

Categories

- category_id (PK)
- category_name (e.g., Road, Streetlight, Drainage, Sign)
- description

Reports

- report_id (PK)
- user_id (FK → Users)
- category_id (FK → Categories)
- assigned_staff_id (FK → Users, nullable)
- title
- description
- photo_url
- latitude
- longitude
- status (Pending / Under Review / In Progress / Resolved)
- date_submitted
- date_updated

Report_Updates

- update_id (PK)
- report_id (FK → Reports)
- updated_by (FK → Users)
- update_type
- previous_status
- new_status
- details
- created_at

Notifications

- notification_id (PK)
- user_id (FK → Users)
- report_id (FK → Reports)
- message
- is_read
- date_created

Relationships

- One **User** (Customer) can submit many **Reports** (1-to-many)
- One **Category** can be linked to many **Reports** (1-to-many)
- One **Report** can have many **Report_Updates** entries (1-to-many)
- One **Staff User** can be assigned to many **Reports** (1-to-many)
- One **Report** can have many **Report_Updates** (1-to-many)
- One **User** can receive many **Notifications** (1-to-many)

VII. Technology Stack

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	Node.js with Express.js
Database	PostgreSQL
Maps/Geotagging	Leaflet.js with OpenStreetMap or Google Maps API
Authentication	JWT-based authentication with Role-Based Access Control (Admin/Staff/Customer)
Image Storage	Cloudinary or local server storage (Multer)
Notifications	In-app notifications; email via Nodemailer or Resend.com (optional)
Hosting/Deployment	Vercel/Netlify (frontend), Render/Railway (backend)
Version Control	Git + GitHub

The selected technology stack establishes the architectural foundation for building a responsive, scalable, and secure public infrastructure reporting system. By utilizing **React.js** and **Tailwind CSS** for an accessible frontend, **Node.js with Express.js** for an optimized backend API, and **PostgreSQL** for reliable database storage with third-party services like Supabase, the architecture guarantees smooth end-to-end communication across the presentation, application, and data layers. Specialized services, including **JWT** for role-based access control, **Leaflet.js/Google Maps** for interactive geotagging, **Cloudinary/Multer** for media storage, and automated deployment services, provide the technical capabilities needed to effectively deliver real-time report tracking and community transparency efficiently.

References:

- Afonso, W. (2017, April 24). *Crowdsourcing for local governments: Research Review – Death & taxes.*
<https://deathandtaxes.sog.unc.edu/2017/04/24/crowdsourcing-for-local-governments-research-review/#:~:text=They%20conclude%20with%20three%20themes%20they%20believe,help%20and%20feel%20like%20they%20can%20help.>
- Anand, A., Acharya, A., & Ja, A. N. (2026). *A research paper on Crowdsourced Civic Issue Reporting and Resolution System. Zenodo (CERN European Organization for Nuclear Research).* <https://doi.org/10.5281/zenodo.19677894>